

Technische Informatik

Einführung

AUTOR

Prof. Johannes Baumgart  

ZUGEHÖRIGKEIT

Hochschule Esslingen

VERÖFFENTLICHT

21. Oktober 2024

1 Virtualisierung mit Docker

1.1 Lernziele

- Die Unterschiede zwischen dedizierten Hosts, virtuellen Maschinen und Docker-Installern/-Installation
- Der Docker-Befehl: ua container starten, ...
- Erzeugen von Docker Images

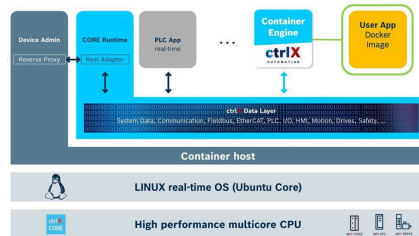
1.2 Motivation

WFM = „Works for me“: Jeder hat schon mal eine Programmierumgebung aufgesetzt und aus irgendeinem Grund hat es nicht funktioniert. Dazu kommt noch, dass es unterschiedliche Betriebssysteme gibt, auf denen dann wieder andere Schritte durchgeführt werden müssen, um eine bestimmte Software laufen zu lassen.

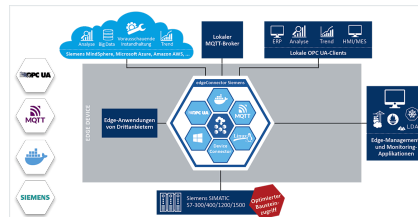
Bei dem **Git-Flow-Ansatz** ist genau diese Komplexität abgebildet, dass es für jedes Release einen Test-Branch gibt, damit insbesondere diese unterschiedlichen Betriebsumgebungen abgesichert werden können.

In diesen Fällen kann eine Virtualisierung helfen.

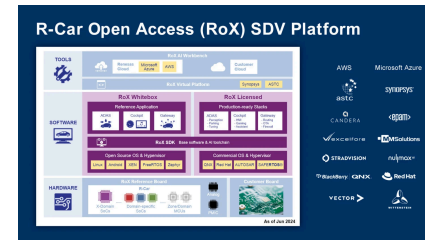
1.3 Einsatzgebiete



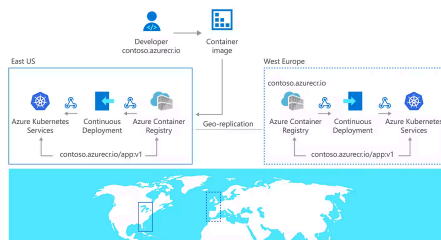
(a) Bosch Rexroth ctrlx mit Docker Engine



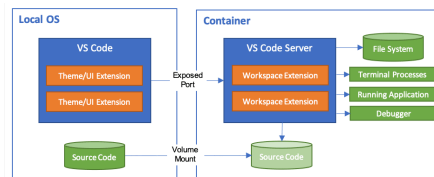
(b) Softing Industrial Automation GmbH Edge Device mit Siemens SIMATIC SPS



(c) Anwendung von Containern im Auto



(d) Beispiel für die Georeplikation von Microsoft Azure



(e) Docker als Entwicklungsumgebung

Abbildung 1: Beispielanwendungen von SPS (Edge), SDV-Architektur (In-Car) bis Azure (Cloud) Anwendungen.

Anwendungen ([Abbildung 1](#))

- Auf SPS Umgebungen (Bosch, Festo) - [Abbildung 1 \(a\)](#) und [Abbildung 1 \(b\)](#)
- Wolkenumgebungen - [Abbildung 1 \(d\)](#)
- Zur Entwicklung - [Abbildung 1 \(e\)](#)
- Auf Embedded Hardware (Fahrzeug Steuergeräte) - [Abbildung 1 \(c\)](#)

1.4 Virtualisierung: Virtuelle Maschinen- vs. Container-Virtualisierung

Bei der Desktop-Virtualisierung lassen sich grundsätzlich zwei Varianten unterscheiden. Beide basieren auf der zugrunde liegenden Hardware (wie Laptop oder Desktop-PC) und dem darauf installierten (Host-) Betriebssystem (Linux, FreeBSD, macOS, Windows usw.). Auf diesem Fundament läuft die Virtualisierungssoftware (vgl. [Abbildung 2](#) (Links))

Die zugrundeliegende Hardware besteht notwendigerweise aus einem Prozessor. Grundsätzlich haben sich in den letzten Jahren verschiedene Prozessorarchitekturen etabliert:

- ARM
- X86
- AMD64
- Apple-Silizium

Diese Prozessoren haben unterschiedliche Befehlssätze. Daraus ergibt sich die Notwendigkeit, dass das Programm auf dem jeweiligen Befehlssatz kompiliert sind.

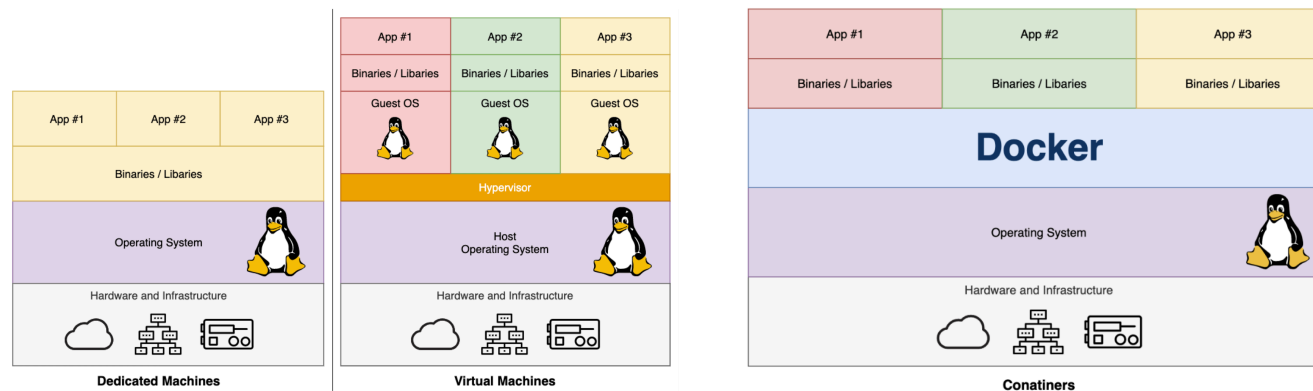


Abbildung 2: Übersicht von unterschiedlichen Virtualisierungskonzepten

Die [Figure 2](#) (mitte) zeigt eine traditionelle **Virtualisierung mit virtuellen Maschinen (VMs)**. In einer VM wird ein vollständiges Betriebssystem (das sogenannte „Gast-Betriebssystem“) installiert, in dem anschließend die gewünschte Anwendung ausgeführt wird. Die Virtualisierungssoftware (wie VirtualBox oder VMware) läuft hierbei als Anwendung auf dem Host-Betriebssystem und stellt der VM eine virtuelle Maschine mit CPU, RAM und anderen Ressourcen zur Verfügung. Die Systemaufrufe innerhalb der VM werden in Aufrufe des Host-Betriebssystems übersetzt.

Dieser Ansatz benötigt jedoch erhebliche Ressourcen:

- Da in der VM ein vollständiges Betriebssystem läuft, ist die entsprechende Datei oft mehrere Dutzend Gigabyte groß.
- Zusätzlich wird Hardware für die Übersetzung beansprucht, was CPU-Leistung und RAM kostet.
- Das Starten einer VM ist aufgrund des vollständigen Hochfahrens des Gast-Betriebssystems entsprechend zeitaufwendig.
- Der Vorteil ist jedoch, dass die Prozesse in einer VM stark isoliert sind, sodass sie als „Sandbox“ fungieren. Dies erschwert es Schadsoftware, aus der VM auszubrechen und das Host-Betriebssystem zu gefährden.

Die [Figure 2](#) (rechts) zeigt eine ressourcenschonendere **Virtualisierung auf Containerbasis**. Hier laufen die Anwendungen direkt als Prozesse im Host-Betriebssystem, sodass kein vollständiges Gast-Betriebssystem erforderlich ist. Techniken wie namespaces und cgroups (in Linux und FreeBSD verfügbar) sorgen dafür, dass die Container-Prozesse vom Rest des Systems abgeschottet sind und andere Prozesse auf dem Host nicht „sehen“.

- Container werden beispielsweise von Docker verwaltet und als Dateien im Dateisystem des Hosts gespeichert.
- Sie benötigen wesentlich weniger Speicherplatz als VMs, starten schneller und nutzen die Hardwareressourcen effizienter.

Ein Nachteil ist,

- dass in der Regel ein Linux-Host erforderlich ist (unter Windows wird der Linux-Layer WSL verwendet;

- für macOS wurde ursprünglich eine Linux-VM genutzt, mittlerweile jedoch eine eigene schlanke Virtualisierung).

Zudem bieten Container meist kein grafisches Benutzerinterface, und da die Prozesse direkt im Host-Betriebssystem ausgeführt werden, gibt es keine zusätzliche Sicherheitsisolation wie bei einer VM.

Achtung bei unterschiedlichen Hardware Architekturen

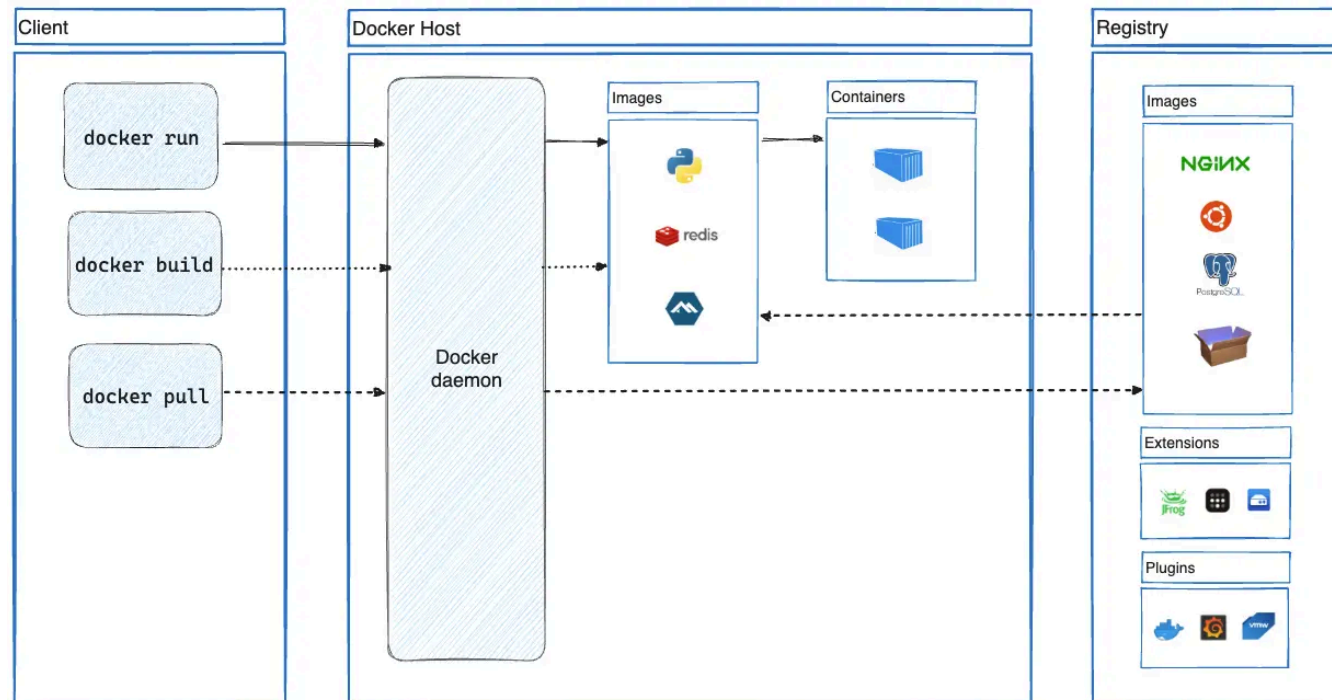
In jedem Fall muss die Hardwarearchitektur berücksichtigt werden: Auf einem Intel-System lassen sich in der Regel keine ARM-basierten VMs oder Container ausführen und umgekehrt.

1.5 Docker (eine Container Virtualisierungslösung)

Was ist Docker (informal)?

Docker ist eine offene Plattform zur Entwicklung, Bereitstellung und Ausführung von Anwendungen. Sie ermöglicht die Trennung von Anwendungen und der zugrunde liegenden Infrastruktur, was eine beschleunigte Bereitstellung von Software erlaubt. Mithilfe von Docker können Infrastrukturen auf ähnliche Weise verwaltet werden wie Anwendungen. Durch die Anwendung der Docker-Methodik für Codeauslieferung, Testprozesse und Bereitstellung wird die Zeitspanne zwischen der Codeerstellung und der Produktivsetzung signifikant verkürzt.

1.5.1 Architektur

Figure 3: Docker Architektur¹

Docker verwendet eine Client-Server-Architektur. Der Docker-Client kommuniziert mit dem Docker-Daemon, der die schwere Arbeit des Erstellens, Ausführens und Verteilens Ihrer Docker-Container übernimmt. Der Docker-Client und der Daemon können auf demselben System ausgeführt werden, oder Sie können einen Docker-Client mit einem entfernten Docker-Daemon verbinden. Der Docker-Client und der Daemon kommunizieren über eine REST-API, über UNIX-Sockets oder eine Netzwerkschnittstelle. Ein weiterer Docker-Client ist Docker Compose, mit dem Sie mit Anwendungen arbeiten können, die aus einer Reihe von Containern bestehen.

Begriffe

- **Docker-File:** Beschreibungsdatei, wie Docker ein Image erzeugen soll.
- **Image:** Enthält die Dinge, die lt. dem Docker-File in das Image gepackt werden sollen. Kann gestartet werden und erzeugt damit einen Container.
- **Container:** Ein laufendes Images (genauer: eine laufende Instanz eines Images). Kann dann auch zusätzliche Daten enthalten.

- **Image Registry:** z.B. hub.docker.com/search
- Der **Docker-Daemon** (`dockerd`) lauscht auf Docker-API-Anforderungen und verwaltet Docker-Objekte wie Images, Container, Netzwerke und Volumes. Ein Daemon kann auch mit anderen Daemons kommunizieren, um Docker-Dienste zu verwalten.

Praxiszusammenfassung

Ein docker image ist ein immutable snapshot des Dateisystems.

Ein docker container ist

- ein temporäres Dateisystem
- ein Netzwerk stack
- eine Prozessgruppe

1.5.2 Container Life Cycle und Befehle zur Manipulation

In [Figure 4](#) werden die verschiedenen Status eines Containers in Bezug zu den Commandos gesetzt, um den Status zu manipulieren.

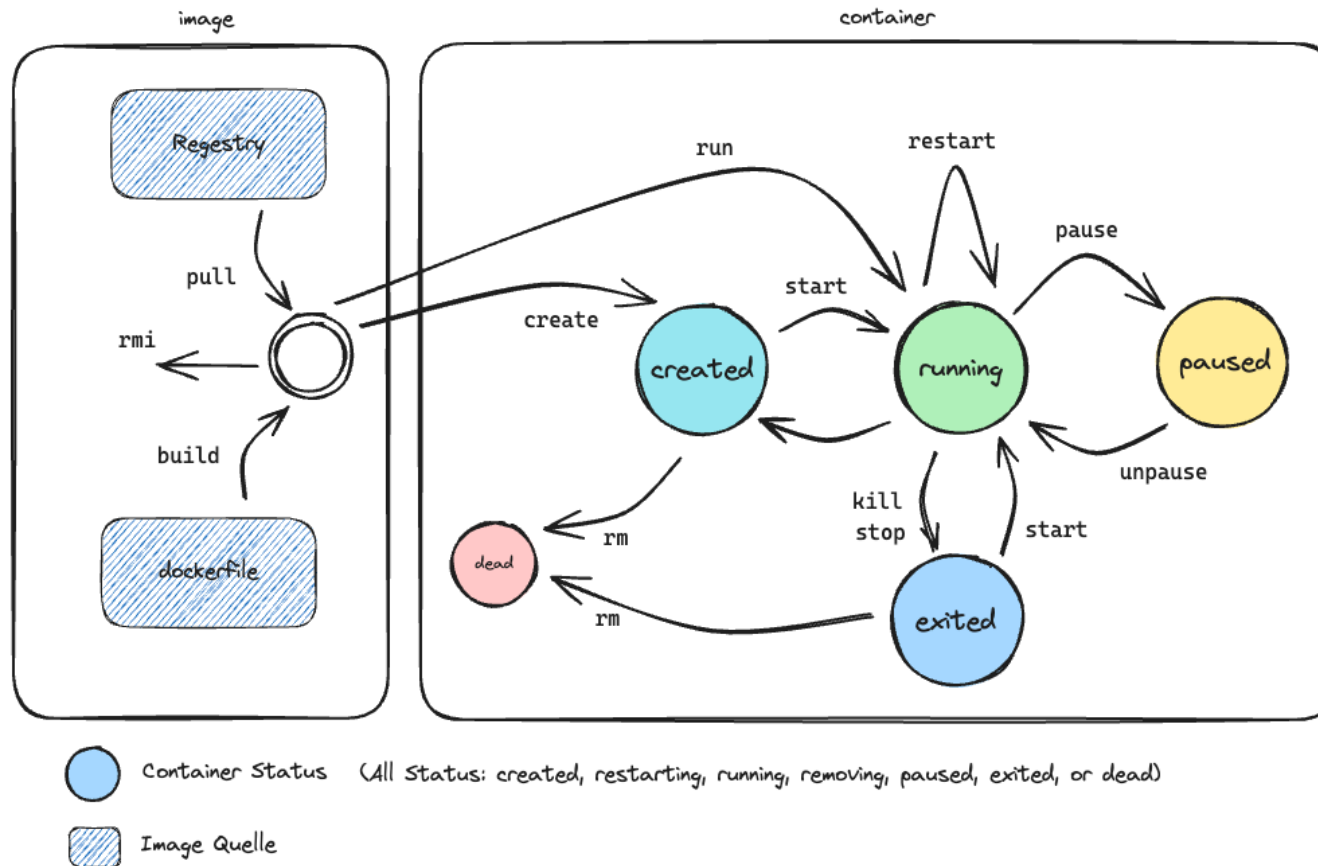


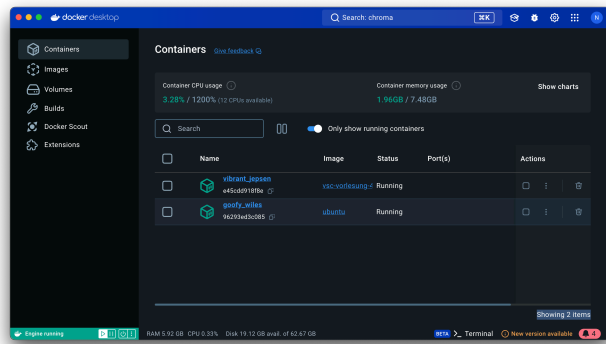
Figure 4: Docker Commands und Container Status

1.5.2.1 docker run

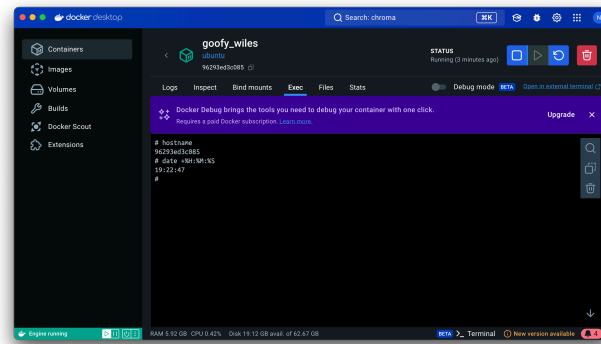
`docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

- das Dateisystem wird von dem image `IMAGE` initialisiert
- Optional kann ein bestimmtes Kommando mit Argumenten innerhalb des Containers ausgeführt werden `[COMMAND] [ARG...]`

Beispiele:



(a) container status



(b) container command exec

Figure 5: Beispiel in der docker gui

Beispiele auf der Kommandozeile

- Abfragen des Host-Namens im container:

```
docker run ubuntu hostname
```

- Abfragen der aktuellen Zeit:

```
docker run ubuntu date +%H:%M:%S
```

Hilfe für die docker CLI

Mit der **OPTION** gleich `--help` kann man sich die weiteren Optionen anzeigen lassen. Dies gilt für alle **docker** CLI Befehle.

Wichtige Optionen ([OPTIONS]):

1.5.2.1.1 Option `-d` detached

Option `-d` detached:

- Die Standardoption ist, dass der Container im Vordergrund gestartet wird.
 - `stdout` und `stderr` werden an das host-Terminal weitergeleitet.
 - `docker run` reicht die exit codes an den Hauptprozess weiter.
- Mit `-d` (d=detached) wird der container entkoppelt gestartet und `docker run` kehrt zurück. Auf dem host-Terminal wird die ID des gestarteten Containers ausgegeben.

Beispiel:

```
docker run -d ubuntu date +%H:%M:%S  
f0d8409f03b23acc0795879055bb691865be53f96b2e49f72dfa5a239d1a2112
```

Wielange bleibt der Container im Status `running` ?

Wenn der Standardprozess terminiert wird auch der Container wieder beendet.

Aufgabe: Führen sie die beiden Befehle aus und untersuchen sie den Status der Container.

Untersuchung mit `docker ps` und mit der Docker GUI der beiden run-Befehle:

1. Container wird direkt nach der Ausgabe beendet:

```
docker run -d ubuntu hostname
```

2. Container wird nicht beendet, weil bash auf Benutzereingaben wartet:

```
docker run -d ubuntu bash
```

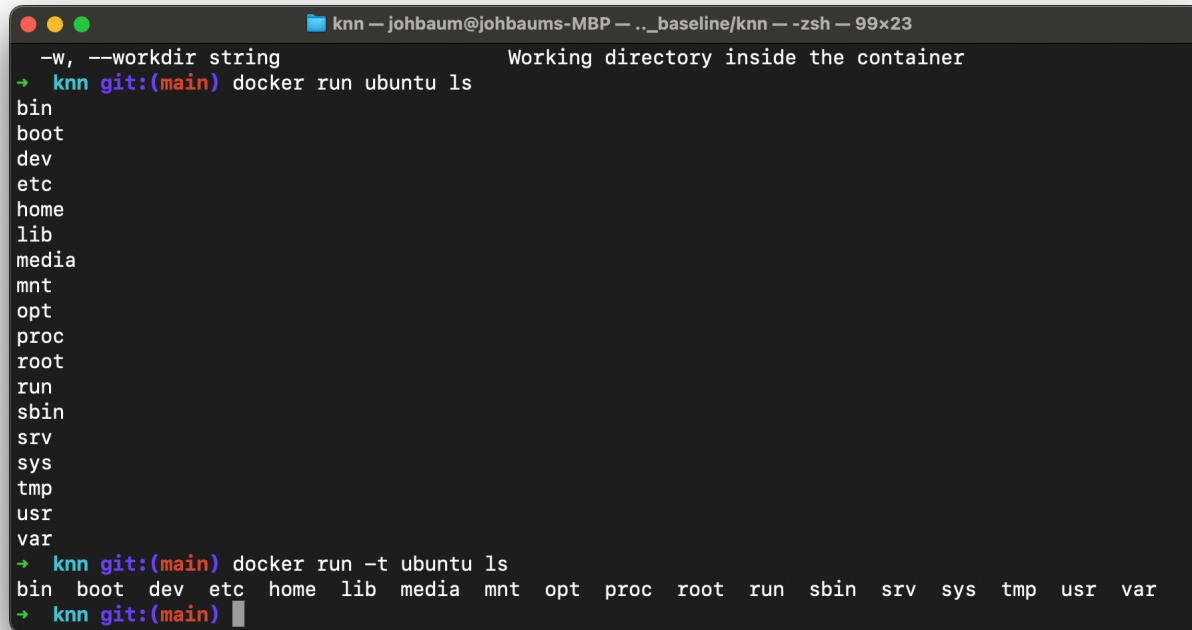
1.5.2.1.2 Option `-t` bzw. `--tty`

Option `-t` bzw. `--tty` Ausführen in einem Pseudo-Terminal im Container:

Die **Abkürzung** `tty` des englischen Worts für Fernschreiber „teletype“ ist noch heute in Unix- und Linux Betriebssystemen und entsprechender Software allgegenwärtig.²

Vergleichen Sie folgende Befehle

1. Ohne die `-t` Option: `bash` `docker run ubuntu ls`

2. Mit der `-t` Option: `bash` `docker run -t ubuntu ls`

```
knn — johbaum@johbaums-MBP — .._baseline/knn — zsh — 99x23
-w, --workdir string      Working directory inside the container
→ knn git:(main) docker run ubuntu ls
bin
boot
dev
etc
home
lib
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
→ knn git:(main) docker run -t ubuntu ls
bin boot dev etc home lib media mnt opt proc root run sbin srv sys tmp usr var
→ knn git:(main)
```

Figure 6: Unterschied in der Ausgabe mit der Option `-t`

Mit der Option werden Signale nicht an `docker run` weiter geleitet. Es werden Signale (wie `^C`) nicht an den Container weitergeleitet. Beispielsweise detached `^C` nur den Container. Damit ist `stdin` nicht mehr verwendbar.

In [Figure 7](#) wird das Verhalten der beiden Optionen `-t` und `-t -i` beispielhaft gegenübergestellt.

```
johbaum — johbaum@johbaums-MBP — zsh — 46x9
~
→ ~ docker run -t ubuntu bash
root@691a9822f107:/# date
^Ccontext canceled
→ ~
```

Nur mit Option `-t` wird `stdin` direkt geschlossen und ist nicht benutzbar

```
johbaum — johbaum@johbaums-MBP — zsh — 46x9
~
→ ~ docker run -t -i ubuntu bash
root@a90de1a893af:/# date
Wed Oct 2 05:51:14 UTC 2024
root@a90de1a893af:/# ^C
root@a90de1a893af:/#
exit
→ ~
```

Nur mit Option `-t -i` wird `stdin` nicht geschlossen und ist benutzbar. Mit `^D` wird der `bash`-Prozess geschlossen.

Figure 7: Unterschiedliches Verhalten des Terminals mit den Optionen `-t` und `-i`.

Mit folgenden Befehlen und Signalen wird der Container nur detached und `stdin` wird sofort geschlossen. Damit wird die Ausgabe von `date` nicht angezeigt:

```
→ ~ docker run -t ubuntu bash
root@1862df230b26:/# date
^C
→ ~
```

Nach dem Signal `^C` bleibt der Container im Status `running` und ist detached.

1.5.2.1.3 Mit der Option `-i` oder `--interactive` Interactive bleibt `stdin` verwendbar.

Mit folgenden Befehlen und Signalen wird das Signal an den Container weitergeleitet und in diesem Fall auf den `bash`-Prozess angewendet.

```
→ ~ docker run -t -i ubuntu bash
root@582c8c45c825:/# date
Wed Oct 2 05:42:19 UTC 2024
root@582c8c45c825:/# ^C
```

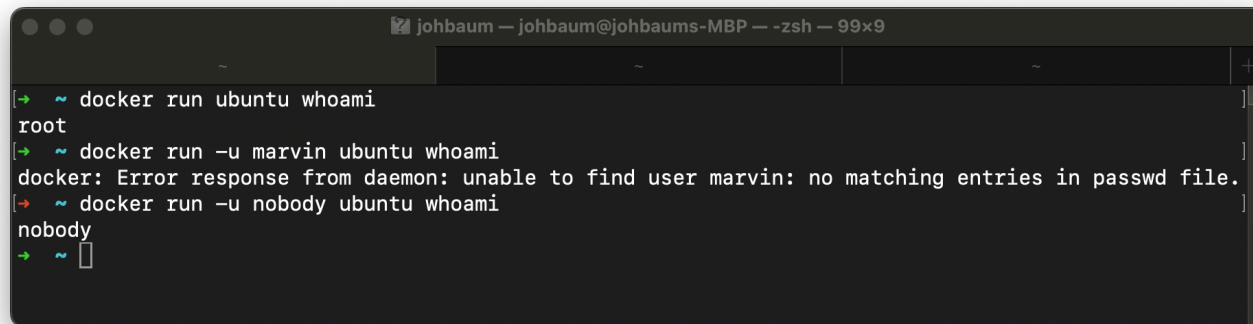
```
root@582c8c45c825:/# ^D
→ ~
```

Nach dem Signal `^D` wird der `bash`-Prozess beendet und damit auch der Container.

Mit der Option `-u ( )` lässt sich der Container mit einem bestimmten Benutzer starten (vgl. [Figure 8](#)).

Mit dem Befehl `whoami` wird der aktuelle Nutzer ausgegeben.

Der Nutzer `root` ist der User mit umfassenden Administratorrechten.



```
johbaum — johbaum@johbaums-MBP — zsh — 99x9
~
→ ~ docker run ubuntu whoami
root
→ ~ docker run -u marvin ubuntu whoami
docker: Error response from daemon: unable to find user marvin: no matching entries in passwd file.
→ ~ docker run -u nobody ubuntu whoami
nobody
→ ~
```

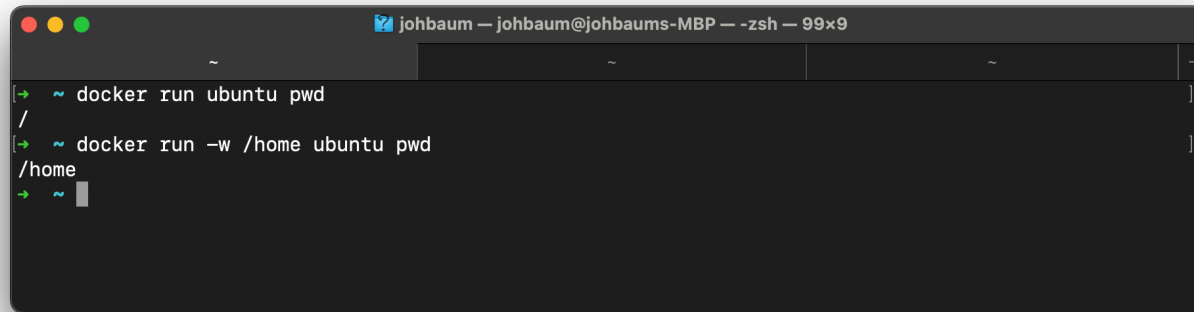
Figure 8: Unterschiedliche User auswählen - `admin` und `nobody` sind verfügbare User `marvin` ist im Container nicht verfügbar

Wenn ein User nicht existiert, dann wird eine Fehlermeldung ausgegeben und der User wird nicht gesetzt.

1.5.2.1.4 Option `-w`

Mit der Option `-w` lässt sich das working directory / Arbeitsverzeichnis wechseln (vgl. [Figure 9](#)).

Der Befehl `pwd` zeigt den absoluten Pfad zum aktuellen Verzeichnis / Ordner an.



```
johbaum — johbaum@johbaums-MBP — zsh — 99x9
[→ ~ docker run ubuntu pwd
/
[→ ~ docker run -w /home ubuntu pwd
/home
[→ ~
```

Figure 9: Working Directory festlegen

Wenn das Arbeitsverzeichnis / Working Directory nicht existiert, dann wird es angelegt.

1.5.2.1.5 Option `-e`

Mit der Option `-e` können Umgebungsvariablen gesetzt werden.

Beispiel mit den Umgebungsvariablen `FOO` und `BAR`.

Im der Ausgangssituation sind die Variablen nicht gesetzt:

```
docker run ubuntu sh -c 'echo $FOO $BAR'
```

mit folgendem Befehl werden diese gesetzt:

```
docker run -e FOO=foo -e BAR=bar ubuntu sh -c 'echo $FOO $BAR'
```

Beispiel: Unter Windows ist `$PATH` eine Umgebungsvariable. In dieser sind alle Verzeichnisse vermerkt, in der Programme zu finden sind.

1.5.2.1.6 Option `-h`

Ausgangssituation:

```
docker run ubuntu hostname
```

Mit der Option `-h` kann der host Name explizit gesetzt werden:

```
docker run -h my-nice-hostname ubuntu hostname
```

1.5.2.1.7 Option `--name`

Start eines Containers mit einem zufälligen Namen:

```
docker run -d -t ubuntu
```

Start eines Containers mit dem eindeutigen Namen `myLinux`:

```
docker run -d -t --name myLinux ubuntu
```

Best Practice

Häufig werden die Container in Applikationen, die aus mehreren Containern besteht, danach benannt, welche Prozess und damit Funktionen sie zur Verfügung stellen.

Den Container Namen kann man statt der Container ID verwenden. Beispiele sind:

```
docker run --name date-ctr ubuntu date
```

Starten über den Container Namen

```
docker start date-ctr
```

In den Logs sollten jetzt 2 Einträge verfügbar sein:

```
docker logs date-ctr
```

Und der Container kann natürlich auch wieder gelöscht werden:

```
docker rm date-ctr
```

1.5.2.1.8 Weitere Optionen und Idiome

Mit der Option `--rm` wird der Container direkt nachdem er den `running` Zustand verlässt automatisch gelöscht.

Damit lässt sich ein "throwaway" container starten. z.B.

```
docker run --rm -t -i ubuntu
```

Mit `docker ps -a` werden alle container aufgelistet, die in dem System verfügbar sind.

Mit `docker container prune` werden alle gestoppten Container entfernt.

1.5.2.2 docker logs

Mit `docker logs ID` kann die Ausgabe des Containers auf `stdout` und `stderr` auch von detachedten Containern angezeigt werden.

Beispiel in [Figure 10](#) sind die Logs für unseren Container mit der Ausgabe der aktuellen Zeit abgebildet. der Container wurde mit folgendem Befehl mehrmals gestartet:

```
docker run -d ubuntu hostname
```

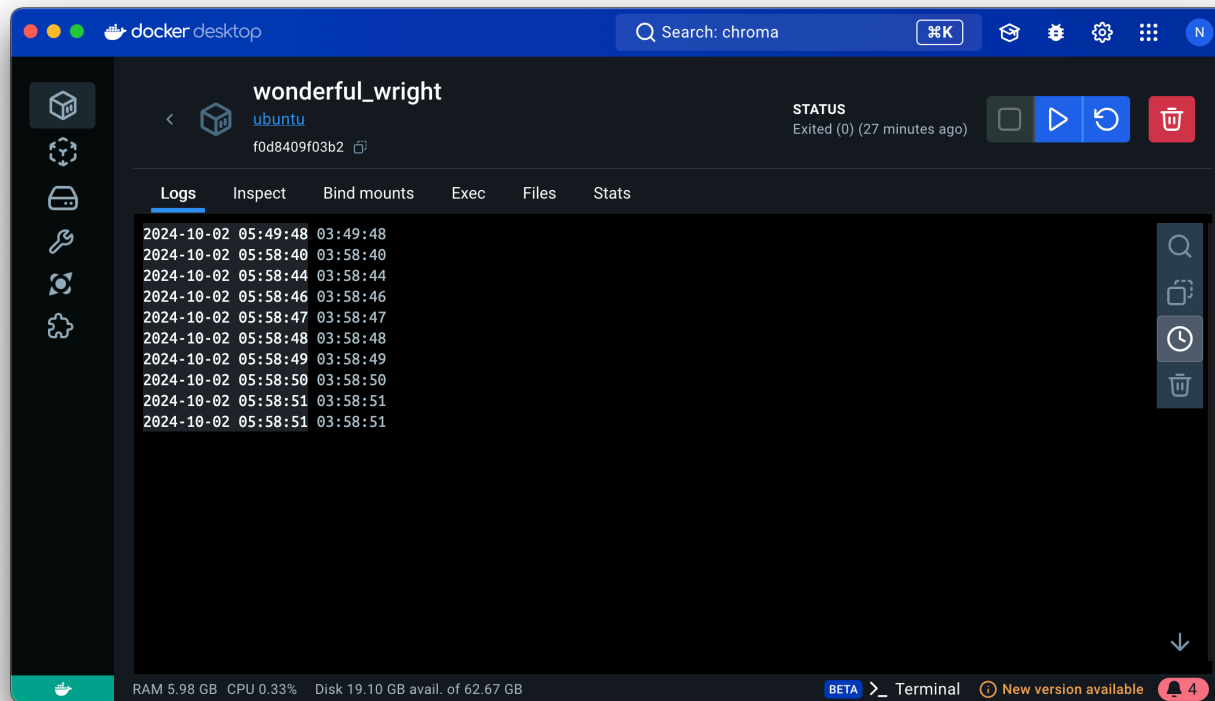



Figure 10: Solange der Container nicht gelöscht wird bleiben die logs der vorherigen Ausführungen erhalten

Aufgabe Zeigen Sie sich mit `docker logs` die logs des Containers an

1. Container ID mit `docker ps` herausfinden.
2. Mit `docker logs` die Ausgabe auf stdout anzeigen.

1.5.2.3 Weitere docker cli Befehle und der vollständige Event-Prozess

Weitere Befehle und Idiome finden Sie in dem [Cheat Sheet](#).

Die [Figure 4](#) zeigt eine vereinfachte Darstellung mit Fokus auf die Verbindung zu den CLI-Kommandos. In [Figure 4](#) sieht man die die unterschiedlichen Docker Events und die Status-Übergänge.

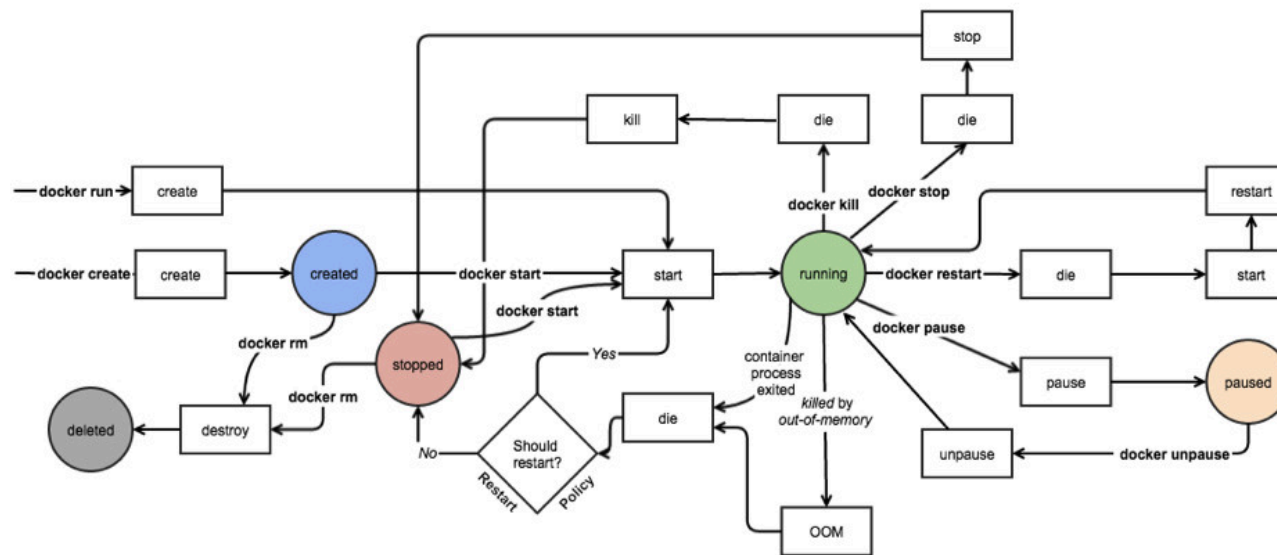


Figure 11: Docker Events und Status - Details

1.5.3 Data Volumes

Es ist möglich ein externes Dateisystem (z.B. vom Host) in den Docker Container zu mounten. Damit kann der Container auf die Dateien zugreifen und diese manipulieren. In [Figure 12](#) werden die 3 Möglichkeiten dargestellt.

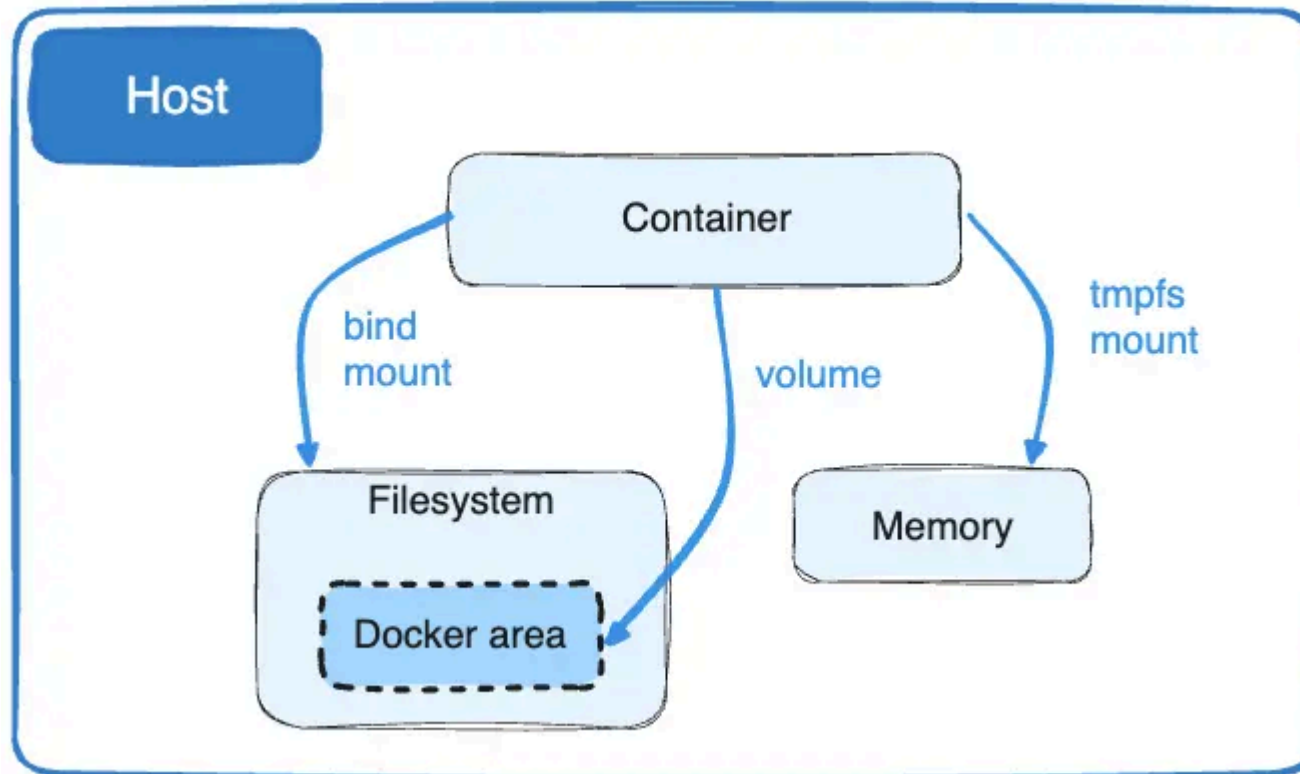


Figure 12: Docker Volume Mount Möglichkeiten: bind mount (1) und volume (2) für persistente Daten und tmpfs mount (3) für nicht persistente Daten

Damit sind die Dateien nicht Teil des Containers oder des Images und somit nicht von der Lifetime der Docker Elemente abhängig. Weiter können die Dateien auch außerhalb des Containers verändert werden.

Es gibt 2 Varianten, um ein persistentes Dateisystem zu mounten: mit der Option `-v` (Volumes) und der Option `--mount` (Mounts).

Vorteile von `-v` sind:

- Volumes

Syntax mit `-v`:

```
docker run -v /hostpath:/ctrpath[:ro] ...
```

- Pfad auf dem Host Rechner (z.B. Laptop): `/hostpath`
- Pfad innerhalb des Containers: `/ctrpath` (wird automatisch erstellt, falls der Pfad nicht existiert.)
- Mit dem Zusatz `[:ro]` kann der Container die die Dateien nur lesen.

Unterschied zwischen `-v` und `--mount` ³

Wenn Sie `-v` oder `--volume` verwenden, um eine Datei oder ein Verzeichnis zu binden, das noch nicht auf dem Docker-Host existiert, erstellt `-v` den Endpunkt für Sie. Er wird immer als Verzeichnis erstellt.

Wenn Sie `--mount` verwenden, um eine Datei oder ein Verzeichnis zu binden, das noch nicht auf dem Docker-Host vorhanden ist, erstellt Docker es nicht automatisch für Sie, sondern generiert eine Fehlermeldung.

Mount Übung

öffnen sie ein Terminal und erstellen Sie einen leeren Ordner

```
mkdir ./host-persistent
```

1. Starten Sie den Container mit

```
docker run --rm -t -i -v ./host-persistent:/persistent ubuntu
```

2. Erzeugen Sie eine Datei `zitat.txt` innerhalb des Containers im Ordner `/persistent` und Schreibe einen Text in die Datei. z.B. mit

```
echo "Macht's gut und danke für den vielen Fisch. - Die Delfine" >/persistent/zitat.txt
```

3. Beende die Container mit `exit` oder `^D` schaue den Inhalt von `zitat.txt` auf dem PC an.
4. Starte den Container erneut (aber mit readonly) und schaue die Datei im Container an.
5. Versuche die Datei erneut zu verändern (siehe 2.). Was wird auf der Console ausgegeben?

1.5.4 Named Volumes

Mit dem CLI Kommando `docker volume COMMAND` kann Docker Volumes außerhalb der Container verwalten.

Diese Volumes werden jetzt von Docker verwaltet und nicht mehr nur vom Betriebssystem des Hosts (wie bei den Mounts in [Section 1.5.3](#)).

1. Erstellen eines Volumes mit dem Namen `my-volume`:

```
docker volume create my-volume
```

2. Anzeigen der verwalteten Volumes

```
docker volume ls
```

3. Volumen in einem Container verwenden. Anwendungsbeispiel:

```
docker run --rm -t -i -v my-volume:/vol ubuntu sh -c "echo 'Ein Big Mac ist ein Big Mac,
aber die nennen ihn Le Big Macke.' > /vol/zitat.txt"
```

4. Mit folgendem Kommando kann man die Details des Volumes untersuchen:

```
docker volume inspect my-volume
```

5. Um zu untersuchen, ob das Zitat tatsächlich in der Datei steht, können wir wieder einen temporären Container starten und den Inhalt der Datei auf der Console ausgeben:

```
docker run --rm -t -i -v my-volume:/vol ubuntu cat /vol/zitat.txt
```

6. Wenn man das Volume wieder löschen möchte:

```
docker volume rm my-volume
```

1.5.5 Ausblick Externes Device im Container verfügbar machen

Es ist möglich auch Devices wie Kameras einem Container zur Verfügung zu stellen. Beispielsweise unter einem Linux-Host (ubuntu), wenn die Kamera unter `/dev/video0` verfügbar ist:

```
docker run --rm -it --entrypoint /bin/bash --device /dev/video0 jrottenberg/ffmpeg  
ffmpeg -f video4linux2 -list_formats all -i /dev/video0
```

1.5.6 Container networking: Kommunikation mit Containern und zwischen Containern

Standardmäßig werden neue Container mit dem main network "bridge" (172.17.0.0/16) verbunden.

1.5.6.1 Kommunikation zwischen Containern

Container sind standardmäßig mit einem Netzwerk ausgestattet und können ausgehende Verbindungen herstellen. Wenn Sie mehrere Container betreiben, die miteinander kommunizieren müssen, können Sie ein benutzerdefiniertes Netzwerk erstellen und die Container mit dem Netzwerk verbinden.

Ebenso wie bei Volumes kann docker Netzwerke verwalten, so dass Container untereinander kommunizieren können.

Erstellen eines Netzwerks mit dem Namen `NETWORK`:

```
docker network create NETWORK
```

Wichtige Eigenschaften von Netzwerken

- Container innerhalb eines Netzwerks können miteinander kommunizieren.
- Wenn ein Container in mehreren Netzwerken ist, dann kann dieser mit beiden Netzwerken gleichzeitig kommunizieren.
- Neue Container werden standardmäßig mit dem Hauptnetz verbunden (namens „bridge“, 172.17.0.0/16)

Ein Container kann zum Start einem Netzwerk hinzugefügt werden:

```
docker run --net=NETWORK
```

Schon gestartete Container (**CONTAINER**) können dynamisch Netzwerken hinzugefügt oder von diesen Entfernt werden.

```
docker network connect NETWORK CONTAINER  
docker network disconnect NETWORK CONTAINER
```

Wenn mehrere Container an dasselbe benutzerdefinierte Netzwerk angeschlossen sind, können sie unter Verwendung der Containernamen als DNS-Hostnamen miteinander kommunizieren. Im folgenden Beispiel wird ein benutzerdefiniertes Netzwerk mit dem Namen my-net erstellt und es werden zwei Container ausgeführt, die mit dem Netzwerk verbunden sind.

Übung zum Nachvollziehen von Netzwerken

1. Welche Container gibt es?
2. Welcher Container kommuniziert wie mit einem anderen Container?
3. Wie heißt das Network?

```
$ docker network create my-net  
$ docker run -d --name web --network my-net nginx:alpine  
$ docker run --rm -it --network my-net busybox  
/ # ping web  
PING web (172.18.0.2): 56 data bytes  
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.326 ms  
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.257 ms  
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.281 ms  
^C  
--- web ping statistics ---  
3 packets transmitted, 3 packets received, 0% packet loss  
round-trip min/avg/max = 0.257/0.288/0.326 ms
```

Lösung zum Nachvollziehen von Netzwerken

1. Den Container mit dem Namen **web** und den 2. Container ohne Namen von dem Image **busybox**.
2. Der 2. Container pinged den Container **web** über dessen gleichnamigen DNS-Namen.

3. Das Netzwerk heißt: `my-net`

1.5.6.2 Einen TCP Port des Containers verfügbar machen

Container werden in einem privaten Netzwerk eingesetzt, sie sind nicht von außen erreichbar (es sei denn, es wird eine Umleitung [eng. redirection] eingerichtet):

```
docker run -p [ipaddr:]hostport:containerport
```

- eingehende Verbindungen an den TCP-Port `hostport` des Hosts werden auf den TCP-Port `Containerport` des Containers weitergeleitet
- Der lauschende Socket bindet sich standardmäßig an 0.0.0.0 (all interfaces) oder an `ipaddr` wenn angegeben.

Übung Port-Weiterleitungen/Redirections

1. In welchem Netzwerk ist der Container in Aufruf 1?
2. Auf welche IP Adresse ist der Container gebunden?
3. Warum ist in Aufruf 2 die IP aus Aufruf 1 nicht mehr verfügbar?

Aufruf 1:

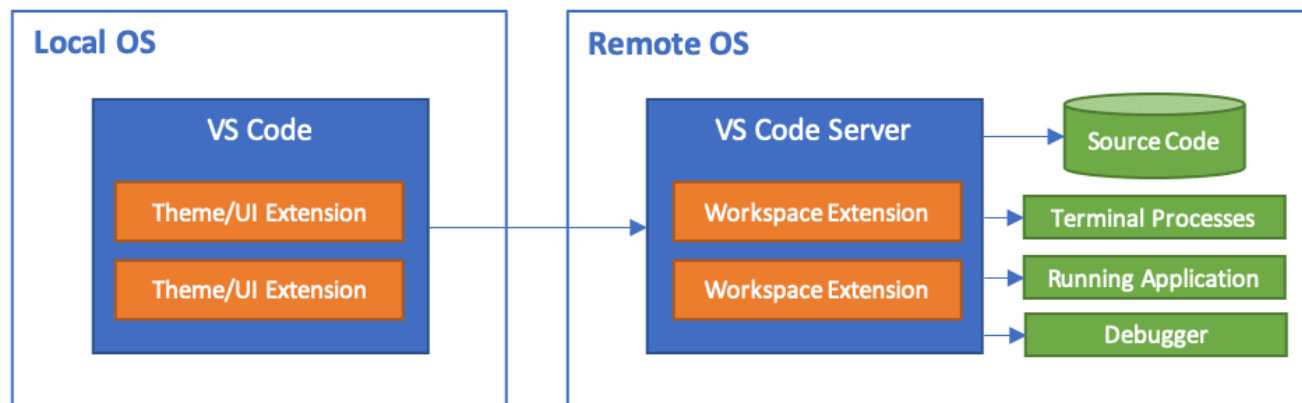
```
$docker run -d -p 80:80 nginx
52c9105e1520980d49ed00ecf5f0ca694d177d77ac9d003b9c0b840db9a70d62
$wget -nv http://localhost/
2024-01-12 18:32:52 URL:http://localhost/ [612/612] -> "index.html" [1]
```

Aufruf 2:

```
$docker run -d -p 127.0.0.1:80:80 nginx
4541b43313b51d50c4dc2722e741df6364c5ff50ab81b828456ca55c829e732c
$wget -nv http://localhost/
2024-01-12 18:37:10 URL:http://localhost/ [612/612] -> "index.html.1" [1]
$wget http://172.17.42.1/
--2024-01-12 18:38:32-- http://172.17.42.1/
Connecting to 172.17.42.1:80... failed: Connection refused.
```


1.6 Devcontainer

Devcontainer sind eine Möglichkeit eine kontrollierte virtuelle Entwicklungsumgebung aufzusetzen, in der die Entwicklung stattfindet:



Dev Container

Hier gibt es ein gutes Tutorial, devcontainer mit VS-Code zum Laufen zu bringen: [devcontainer mit vscode](#)

1.7 Zusammenfassung

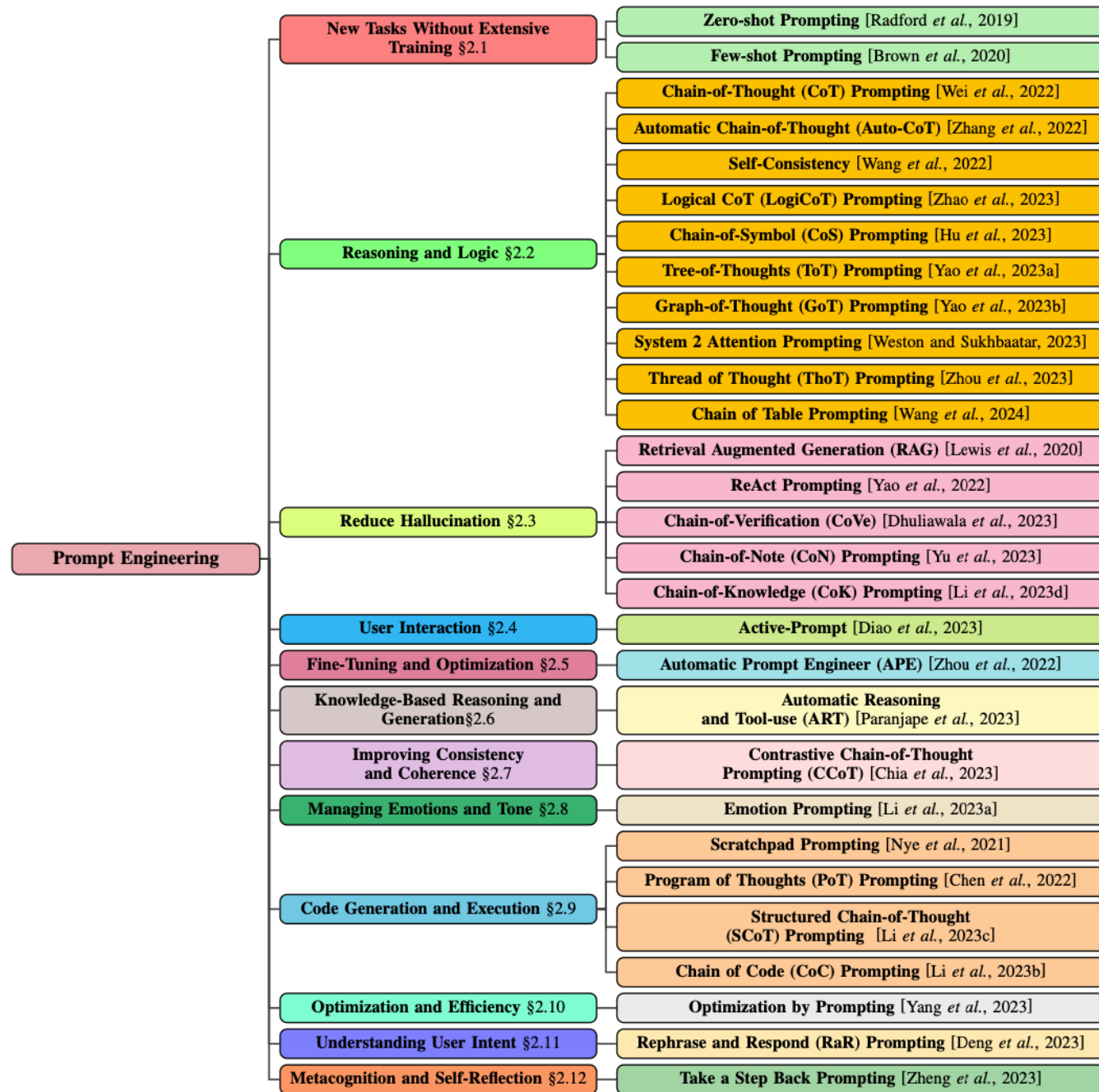
- Docker kann in verschiedenen **Betriebsszenarien** wie SPS-Umgebungen, Cloud-Anwendungen, Embedded-Systemen und **Entwicklungsumgebungen** eingesetzt werden.
- Die Unterschiede zwischen **Virtualisierungen** mit virtuellen Maschinen und Containern werden detailliert erklärt, wobei Container ressourcenschonender und effizienter sind.
- Docker verwendet eine **Client-Server-Architektur**, bei der der Docker-Daemon zentrale Aufgaben wie das Erstellen und Verwalten von Containern übernimmt.

- Verschiedene **Docker-Befehle** wie `docker run`, `-d` (detached) und `-t` (Pseudo-Terminal) werden mit Beispielen und Optionen ausführlich beschrieben.
- Es gibt wichtige Befehle für die **Verwaltung von Containern**, wie etwa `docker ps`, `docker logs` und `docker volume`, die Container- und Volumeninformationen bereitstellen.
- **Docker-Volumes** können gemountet werden, um persistente Daten zwischen Host und Container zu teilen, und Named Volumes können Docker-exklusiv verwaltet werden.
- **Container-Netzwerke** ermöglichen die Kommunikation zwischen Containern, wobei Docker benutzerdefinierte Netzwerke und TCP-Port-Weiterleitungen unterstützt.
- **Devcontainer** bieten eine kontrollierte Entwicklungsumgebung für die Softwareentwicklung, die sich besonders für Projekte mit Visual Studio Code eignet.

2 Short Intro to Prompt Engineering (with Llama models)

Docker example running a local language model and first steps in prompt Engineering.

There are various techniques to build your prompt. Here ist a taxonomy:



Taxonomy of prompt engineering by Sahoo *et al.* (2024)

Here two good frameworks to kick off your prompting:

RGC Prompting

Prompt Formula

- Role - ChatGPT's persona (You are an expert marketer)
- Result - Desired output (Create 5 emails ending with a call to action)
- Goal - Purpose of the output (The goal is to drive sales to my product)
- Context - Who, what, where, why (The emails are for my online audience of entrepreneurs)
- Constraint - Limitations and guidelines (The emails should be friendly and less than 200 words)

Example:

- You are an expert [role]. Create [result]. The goal is [end goal]. The content is for [context]. Your guidelines for writing are [constraints].

I Want You To Act As Prompting

Prompt Formula

1. "I want you to act as a..." (historian, biologist, etc.) This helps to set the stage for ChatGPT's persona.
2. "I will give you..." (my target audience, a specific direction, a product, etc.) This is where you set up the dynamic between you and ChatGPT.
3. "You will then..." (create 5 emails, make a video script, summarize, etc.)
4. "In a ____ tone/style..." (upbeat, professional, tabular format) This is where you explain your desired output along with the personality/style of writing for the output.
5. "The important details are..." (target audience, cost per project, etc.) Be sure to explain all important details and elaborate on specifics from bullet point 2.
6. Refine your output as needed "Be more persuasive" "Put the most important keywords in bold" "Now put this in tabular format"

Example:

I want you to act as a personal coach. I will give you my personal and professional goals. You will then create a 7 day schedule in tabular format for me to follow in order to hit my goals. My short term daily goals are meditate, workout, read, and work on my projects. My long term goals are to sign 30 new clients and save \$10,000 over the next 6 months.

Here is a good starting prompt to answer a question from a document or larger context:

You are an assistant for question-answering tasks. Use the following pieces of retrieved context to answer the question. If you don't know the answer, just say that you don't know. Use three sentences maximum and keep the answer concise.

Question:

{question}

Context:

{context}

Answer:

More practices to enhance your prompts:

#Principle	Prompt Principle for Instructions
1	If you prefer more concise answers, no need to be polite with LLM so there is no need to add phrases like "please", "if you don't mind", "thank you", "I would like to", etc., and get straight to the point.
2	Integrate the intended audience in the prompt, e.g., the audience is an expert in the field.
3	Break down complex tasks into a sequence of simpler prompts in an interactive conversation.
4	Employ affirmative directives such as 'do,' while steering clear of negative language like 'don't'.
5	When you need clarity or a deeper understanding of a topic, idea, or any piece of information, utilize the following prompts: - o Explain [insert specific topic] in simple terms. - o Explain to me like I'm 11 years old. - o Explain to me as if I'm a beginner in [field]. - o Write the [essay/text/paragraph] using simple English like you're explaining something to a 5-year-old.
6	Add "I'm going to tip \$xxx for a better solution!"
7	Implement example-driven prompting (Use few-shot prompting).
8	When formatting your prompt, start with '###Instruction###', followed by either '###Example###' or '###Question###' if relevant. Subsequently, present your content. Use one or more line breaks to separate instructions, examples, questions, context, and input data.
9	Incorporate the following phrases: "Your task is" and "You MUST".
10	Incorporate the following phrases: "You will be penalized".
11	Use the phrase "Answer a question given in a natural, human-like manner" in your prompts.
12	Use leading words like writing "think step by step".
13	Add to your prompt the following phrase "Ensure that your answer is unbiased and avoids relying on stereotypes."
14	Allow the model to elicit precise details and requirements from you by asking you questions until he has enough information to provide the needed output (for example, "From now on, I would like you to ask me questions to ...").
15	To inquire about a specific topic or idea or any information and you want to test your understanding, you can use the following phrase: "Teach me any [theorem/topic/rule name] and include a test at the end, and let me know if my answers are correct after I respond, without providing the answers beforehand."
16	Assign a role to the large language models.
17	Use Delimiters.
18	Repeat a specific word or phrase multiple times within a prompt.
19	Combine Chain-of-thought (CoT) with few-Shot prompts.
20	Use output primers, which involve concluding your prompt with the beginning of the desired output. Utilize output primers by ending your prompt with the start of the anticipated response.
21	To write an essay /text /paragraph /article or any type of text that should be detailed: "Write a detailed [essay/text /paragraph] for me on [topic] in detail by adding all the information necessary".
22	To correct/change specific text without changing its style: "Try to revise every paragraph sent by users. You should only improve the user's grammar and vocabulary and make sure it sounds natural. You should maintain the original writing style, ensuring that a formal paragraph remains formal."
23	When you have a complex coding prompt that may be in different files: "From now and on whenever you generate code that spans more than one file, generate a [programming language] script that can be run to automatically create the specified files or make changes to existing files to insert the generated code. [your question]".
24	When you want to initiate or continue a text using specific words, phrases, or sentences, utilize the following prompt: o I'm providing you with the beginning [song lyrics/story/paragraph/essay...]: [Insert lyrics/words/sentence]. Finish it based on the words provided. Keep the flow consistent.
25	Clearly state the requirements that the model must follow in order to produce content, in the form of the keywords, regulations, hint, or instructions
26	To write any text, such as an essay or paragraph, that is intended to be similar to a provided sample, include the following instructions: o Use the same language based on the provided paragraph/title/text /essay/answer].

Table 1: Overview of 26 randomly ordered prompt principles.

Category	Principles	#Principle
Prompt Structure and Clarity	Integrate the intended audience in the prompt.	2
	Employ affirmative directives such as 'do' while steering clear of negative language like 'don't'.	4
	Use Leading words like writing "think step by step."	12
	Use output primers, which involve concluding your prompt with the beginning of the desired output. by ending your prompt with the start of the anticipated response.	20
	Use Delimiters.	17
Specificity and Information	When formatting your prompt, start with '###Instruction###', followed by either '###Example###' or '###Question###' if relevant. Subsequently, present your content. Use one or more line breaks to separate instructions, examples, questions, context, and input data.	8
	Implement example-driven prompting (Use few-shot prompting).	7
	When you need clarity or a deeper understanding of a topic, idea, or any piece of information, utilize the following prompts: - o Explain [insert specific topic] in simple terms. - o Explain to me like I'm 11 years old. - o Explain to me as if I'm a beginner in [field] - o "Write the [essay/text/paragraph] using simple English like you're explaining something to a 5-year-old."	5
	Add to your prompt the following phrase "Ensure that your answer is unbiased and avoids relying on stereotypes."	13
	To write any text intended to be similar to a provided sample, include specific instructions: o "Use the same language based on the provided paragraph [title/text/essay/answer]."	26
	When you want to initiate or continue a text using specific words, phrases, or sentences, utilize the provided prompt structure: o I'm providing you with the beginning [song lyrics/story/paragraph/essay...]: [Insert lyrics/words/sentence]. Finish it based on the words provided. Keep the flow consistent.	24
	Clearly state the model's requirements that the model must follow in order to produce content, in form of the keywords, regulations, hint, or instructions.	25
	To inquire about a specific topic or idea and test your understanding g. you can use the following phrase [16]: o "Teach me the [Any theorem/topic/rule name] and include a test at the end, and let me know if my answers are correct after I respond, without providing the answers beforehand."	15
	To write an essay/text/paragraph/article or any type of text that should be detailed: o "Write a detailed [essay/text/paragraph] for me on [topic] in detail by adding all the information necessary."	21
	Allow the model to elicit precise details and requirements from you by asking you questions until he has enough information to provide the needed output o "From now on, I would like you to ask me questions to ..."	14
User Interaction and Engagement	To write an essay /text /paragraph /article or any type of text that should be detailed: "Write a detailed [essay/text/-paragraph] for me on [topic] in detail by adding all the necessary information."	21
	To correct/change specific text without changing its style: "Try to revise every paragraph sent by users. You should only improve the user's grammar and vocabulary and make sure it sounds natural. You should maintain the original writing style, ensuring that a formal paragraph remains formal."	22
	Incorporate the following phrases: "Your task is" and "You MUST."	9
	Incorporate the following phrases: "You will be penalized."	10
	Assign a role to the language model.	16
Content and Language Style	Use the phrase "Answer a question given in natural language form" in your prompts.	11
	No need to be polite with LLM so there is no need to add phrases like "please", "if you don't mind", "thank you", "I would like to", etc., and get straight to the point.	1
	Repeat a specific word or phrase multiple times within a prompt.	18
	Add "I'm going to tip \$xxx for a better solution!"	6
	Break down complex tasks into a sequence of simpler prompts in an interactive conversation.	3
Complex Tasks and Coding Prompts	When you have a complex coding prompt that may be in different files: o "From now and on whenever you generate code that spans more than one file, generate a [programming language] script that can be run to automatically create the specified files or make changes to existing files to insert the generated code. [your question]."	23
	Combine Chain-of-thought (CoT) with few-shot prompts.	19

Table 2: Prompt principle categories.

Prompt Engineering Praktiken basierend auf:
Bsharat, Myrzakhan, and Shen (2024)

Setup ollama with llama3.2:4

Terminal 1: Run ollama on PC:

```
docker run -h myOllama ollama/ollama
```

Terminal 2: Load a large Language Model e.g. llama3.2 with 3B parameters:

```
docker exec -it myOllama ollama run llama3.2
```

Or on smaller systems use `llama3.2:1b` with 1B parameters instead.

Alternative Models might be: <https://ollama.com/library/granite3-dense> a small model designed for RAG.

Prompt Engineering⁵

ist ein Konzept der künstlichen Intelligenz (KI), insbesondere der Verarbeitung natürlicher Sprache (englisch: natural language processing, NLP) (Computerlinguistik). Dabei wird ein Prompt, also die Aufgabe, die von der KI erledigt werden soll, in das Eingabefeld geschrieben, z. B. als Frage, statt implizit vorgegeben. Prompt Engineering funktioniert typischerweise, indem es eine oder mehrere Aufgaben in einen prompt-basierten Datensatz umwandelt und ein Sprachmodell mit dem sogenannten „prompt-basierten Lernen“ trainiert.

Die Sprachmodelle GPT-2 und GPT-3 waren wichtige Schritte im Prompt Engineering. Im Jahr 2021 zeigte das Multitask Prompt Engineering unter Verwendung mehrerer NLP-Datensätze eine gute Leistung bei neuen Aufgaben.

Gute Tutorials:

- <https://www.llama.com/docs/how-to-guides/prompting/>
- <https://www.promptingguide.ai/techniques>

Übung - Testen Sie Ihre Eingabetechniken, um eine erste Vorstellung Ihres Laborprojekts zu bekommen

1. Richten Sie Ihren lokalen Ollama-Container ein und installieren Sie darin ein Sprachmodell.
2. Testen Sie einige Eingabeaufforderungen, um herauszufinden, ob Ihre Laborprojektidee funktioniert.

Verweise

Bsharat, Sondos Mahmoud, Aidar Myrzakhan und Zhiqiang Shen. 2024. „Prinzipielle Anweisungen sind alles, was Sie zum Hinterfragen von LLaMA -1/2, GPT -3,5/4 benötigen.“ arXiv. <http://arxiv.org/abs/2312.16171> .

Sahoo , Pranab, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal und Aman Chadha. 2024. „Eine systematische Untersuchung von Prompt Engineering in großen Sprachmodellen : Techniken und Anwendungen .“ arXiv. <http://arxiv.org/abs/2402.07927> .

Fußnoten

1. [docker.docs Erste Schritte](#) 
2. [https://de.wikipedia.org/wiki/Terminal_\(Computer\)](https://de.wikipedia.org/wiki/Terminal_(Computer)) 
3. [docs.docker.com](#) 
4. [ollama – Modellfamilie llama 3.2](#) 
5. [Wikipedia.de – Prompt-Engineering](#) 